PACIFIC INSTITUTE OF TECHNOLOGY — DEPARTMENT OF COMPUTER SCIENCE

CS 545: Advanced Distributed Systems & Cloud Infrastructure

Assignment 1 Master Solution & Official Grading Rubric | Maximum Marks: 50.0

Q1 (12.0 Marks): (a) In the Raft consensus algorithm, state the Log Matching Property and explain how leader election and commit safety rules guarantee that an uncommitted entry from a previous term is never committed directly by counting replicas. (b) Compare steady-state message rounds and leader failure recovery complexity between Multi-Paxos and Raft.

Reference Model Solution:

(a) Raft Log Matching Property: If two logs contain an entry with the same index and term, then they store the same command, and their logs are identical in all entries up through the given index. To prevent committing an entry from an older term, Raft specifies that a leader cannot commit a log entry from an older term simply by counting replicas; it must commit an entry from its CURRENT term by storing it on a majority of servers, which indirectly commits older entries by the Log Matching Property. This rule prevents scenarios where a partitioned former leader overwrites a replicated older entry after network re-convergence. (b) Multi-Paxos vs Raft: In steady state, both Multi-Paxos and Raft require 1 RTT (round-trip time) for log replication once a leader is established. However, during leader failure recovery, Raft simplifies protocol complexity: Raft guarantees that the elected leader already possesses all committed entries (enforced during voting by comparing candidate log term and length), requiring no catch-up phase before accepting writes. In contrast, Multi-Paxos leaders must execute Phase 1 (Prepare/Promise) across all uncommitted slots after election to discover potential values proposed by previous leaders, resulting in significantly higher protocol and recovery complexity.

Required Key Concepts: *Log Matching Property definition (same index and term implies identical history), Leader cannot commit older term entry merely by replica counting; must commit current term entry, Steady-state replication requires 1 RTT for both Multi-Paxos and Raft, Raft leader election invariant: leader already has all committed entries (no catch-up needed), Multi-Paxos requires Phase 1 slot discovery across log slots after leader failure*

Rubric: *Full: Accurate statement of Log Matching Property, clear explanation of current-term commit rule, and rigorous comparison of leader election recovery complexity. | Partial: Explains Raft leader election but omits the current-term commit safety rule or provides superficial Paxos comparison.*

Q2 (13.0 Marks): (a) Contrast Strict 2-Phase Locking (Strict 2PL) and Rigorous 2PL. How does Strict 2PL guarantee conflict serializability and avoid cascading aborts? Explain how deadlocks are detected via the Wait-For Graph (WFG). (b) Describe the Write Skew anomaly under Snapshot Isolation (SI) using a formal execution trace (e.g., the On-Call Doctor problem). How does Serializable Snapshot Isolation (SSI) detect and prevent it?

Reference Model Solution:

(a) In Strict 2PL, all exclusive (write) locks acquired by a transaction must be held until the transaction terminates (commit or abort), whereas shared (read) locks may be released before completion. In Rigorous 2PL, ALL locks (both shared and exclusive) are held until transaction completion. Strict 2PL guarantees serializability because transactions reach a lock point before releasing any write lock, preventing conflicting cycles in the precedence graph. It avoids cascading aborts because no other transaction can read uncommitted dirty data written by an uncommitted transaction. Deadlock Detection: The system maintains a directed Wait-For Graph WFG = (V, E) where vertices V are active transactions and a directed edge T_i -> T_j exists if T_i is waiting for a lock held by T_j. A deadlock exists if and only if the WFG contains a directed cycle (detected periodically using Tarjan's or DFS cycle detection algorithms); the deadlock is broken by aborting a victim transaction. (b) Write Skew under Snapshot Isolation (SI): Consider an invariant: Active Doctors >= 1. Initial state: Doctors D_1=Active, D_2=Active. Transaction T_1 checks active doctors (count=2 >= 1) and sets D_1=Inactive. Concurrently, T_2 checks active doctors on its snapshot (count=2 >= 1) and sets D_2=Inactive. Under SI, since T_1 writes to D_1 and T_2 writes to D_2, their write sets are disjoint, so neither transaction violates the first-committer-wins rule. Both commit, leaving Active Doctors = 0, violating the integrity constraint. Serializable Snapshot Isolation (SSI) detects write skew by tracking rw-antidependency conflicts (where a transaction reads a version that is later overwritten by a concurrent transaction, denoted T_read -rw-> T_write). SSI aborts a transaction if it detects two consecutive rw-antidependencies (T_1 -rw-> T_2 -rw-> T_3) forming a dangerous structure.

Required Key Concepts: *Strict 2PL holds write locks until commit; Rigorous 2PL holds all locks until commit, Strict 2PL eliminates dirty reads, preventing cascading aborts, Wait-For Graph (WFG) cycle detection via DFS/Tarjan to resolve deadlocks, Snapshot Isolation write skew: disjoint write sets allow concurrent commits that violate multi-record constraint, Doctor on-call formal trace showing active doctors dropping to 0, Serializable Snapshot Isolation (SSI) tracks rw-antidependency edges and aborts dangerous structures*

Rubric: *Full: Clear distinction of lock release rules, WFG cycle detection, complete write-skew trace with disjoint write sets, and SSI*

rw-antidependency detection. | Partial: Correct 2PL definitions but vague write-skew trace, or missing SSI explanation.

Q3 (12.0 Marks): (a) Explain Consistent Hashing with Virtual Nodes. If a cluster has N physical nodes and each node is assigned V virtual nodes on a 32-bit hash ring, derive the expected key migration fraction when 1 physical node is added. (b) In Dynamo-style leaderless stores, state the Quorum condition (R, W, N) required to guarantee strong consistency. Explain how Vector Clocks resolve concurrent updates and describe the hazard of Last-Write-Wins (LWW).

Reference Model Solution:

(a) Consistent Hashing maps both keys and node identifiers to positions on a circular hash ring (e.g., [0, 2^32 - 1]). A key is assigned to the first node whose position is >= hash(key). In a basic ring with N nodes, non-uniform hash distribution leads to severe partition imbalance. Virtual Nodes solve this by assigning V distinct points on the ring per physical node. The variance of load across nodes is bounded by O(1/V). When 1 new physical node is added to an existing cluster of N nodes, it claims a fraction 1/(N + 1) of the ring from existing nodes. Therefore, the expected fraction of total keys that must be migrated is exactly 1/(N + 1) (or K / (N + 1) keys for a total of K keys), compared to traditional hash mod N where almost all (N / (N + 1)) keys must be re-hashed. (b) In Dynamo-style leaderless replication with replication factor N, strong (read-your-writes) consistency is guaranteed if and only if the read quorum R and write quorum W satisfy: R + W > N. By the Pigeonhole Principle, the set of nodes read from and the set of nodes written to must overlap by at least one node containing the latest write. Vector Clocks track causal dependencies across nodes: a vector clock VC(a) <= VC(b) implies event 'a' causally preceded 'b'. If neither VC(a) <= VC(b) nor VC(b) <= VC(a), concurrent divergent writes are detected, requiring client-side or application-level reconciliation. Last-Write-Wins (LWW) discards vector clocks and uses physical timestamps (e.g. NTP). Due to inevitable clock drift, skew, and leap seconds, LWW can silently overwrite a newer update with an older one, leading to silent data loss.

Required Key Concepts: *Consistent Hashing ring topology and virtual nodes load balancing (O(1/V) variance), Key migration fraction on node addition is 1 / (N + 1), Quorum condition R + W > N guarantees overlap by Pigeonhole Principle, Vector Clocks detect concurrent conflicts via partial ordering of node tick vectors, Last-Write-Wins (LWW) hazard: physical clock drift/skew silently drops legitimate writes*
Rubric: *Full: Correct key migration derivation (1/(N+1)), quorum condition R+W>N, vector clock causal ordering condition, and LWW clock drift critique. | Partial: States R+W>N but lacks mathematical migration derivation or vector clock conflict explanation.*

Q4 (13.0 Marks): (a) State the completeness and accuracy properties of Chandra-Toueg Eventually Weak Failure Detectors (<>W). How does <>W circumvent the FLP Impossibility result for asynchronous consensus? (b) In Practical Byzantine Fault Tolerance (PBFT), prove why a system of N nodes can tolerate at most f Byzantine nodes if and only if N >= 3f + 1.

Reference Model Solution:

(a) Chandra-Toueg Failure Detectors: <>W (Eventually Weak) is defined by two properties: (1) Weak Completeness: Eventually every crashed process is permanently suspected by some correct process. (2) Eventual Weak Accuracy: There is a time after which some correct process is never suspected by any correct process. The Fischer-Lynch-Paterson (FLP) impossibility result proves that deterministic consensus is impossible in an asynchronous system with even a single crash failure because it is impossible to distinguish a crashed process from an arbitrarily slow network delay. <>W circumvents FLP by augmenting the asynchronous model with minimal timing/synchrony assumptions: during periods of partial synchrony, the detector achieves accuracy, allowing the rotating coordinator algorithm to reach agreement and terminate safely without violating safety during asynchronous periods. (b) PBFT Node Requirement Proof: In an asynchronous distributed system where up to f nodes can be Byzantine (malicious or arbitrarily faulty): Suppose total nodes is N. If f nodes are Byzantine, they may fail to respond altogether (silent crash). Therefore, an algorithm must make progress after receiving responses from N - f nodes (liveness requirement). However, among those N - f responses, up to f of them might be from Byzantine nodes that sent coordinated malicious lies. Thus, the number of guaranteed non-faulty responses is (N - f) - f = N - 2f. To guarantee that the non-faulty correct responses strictly outnumber the faulty ones (safety/correctness), we require: N - 2f > f, which simplifies directly to N > 3f, meaning N >= 3f + 1. By contrast, crash-stop models only require N >= 2f + 1 because crashed nodes cannot fabricate conflicting votes.

Required Key Concepts: *Weak Completeness (every crashed node eventually suspected by some correct node), Eventual Weak Accuracy (eventually some correct node is never suspected), FLP circumvention via partial synchrony and failure detector augmentation, PBFT liveness quorum: must make progress with N - f responses, PBFT safety condition: among N - f responses, at most f can be Byzantine, so N - 2f > f => N >= 3f + 1, Contrast with crash-stop quorum 2f + 1*
Rubric: *Full: Formal statement of Weak Completeness and Eventual Weak Accuracy, FLP circumvention mechanism, and rigorous derivation of N >= 3f + 1. | Partial: Explains PBFT 3f+1 intuitively without formal quorum subtraction, or gives vague failure detector properties.*